Django Application

In Django, a **Project** represents the entire website, while an **Application (App)** represents a **specific feature or module** inside that website.

Example:FlipKart Website

App Name	Responsibility
Authentication	Login, Signup, Logout
Product	Display products
Cart	Add items to cart
Orders	Place and track orders

A **single Django project can contain multiple applications.**

Why Do We Need Applications in Django?

1. Modularity

Each feature is separated into different apps.

Example:

- `user_app`
- `product_app`
- `order_app`

This keeps the project organized.

---

2. Reusability

An app created in one project can be reused in another project.

Example:

- A **Login System App** can be reused in multiple projects.

---

3. Team Collaboration

Different developers can work on different apps.

Example:

- Developer 1 → Authentication App
- Developer 2 → Product App
- Developer 3 → Order App

---

4. Configuration Flexibility

Apps can easily be **enabled or disabled** using `INSTALLED_APPS`.

---

5. Maintainability

If there is a bug, it is easier to **find and fix it inside the specific app** instead of searching the entire project.

Creating an Application

Run this command inside the Django project folder:

python manage.py startapp application_name

Example:

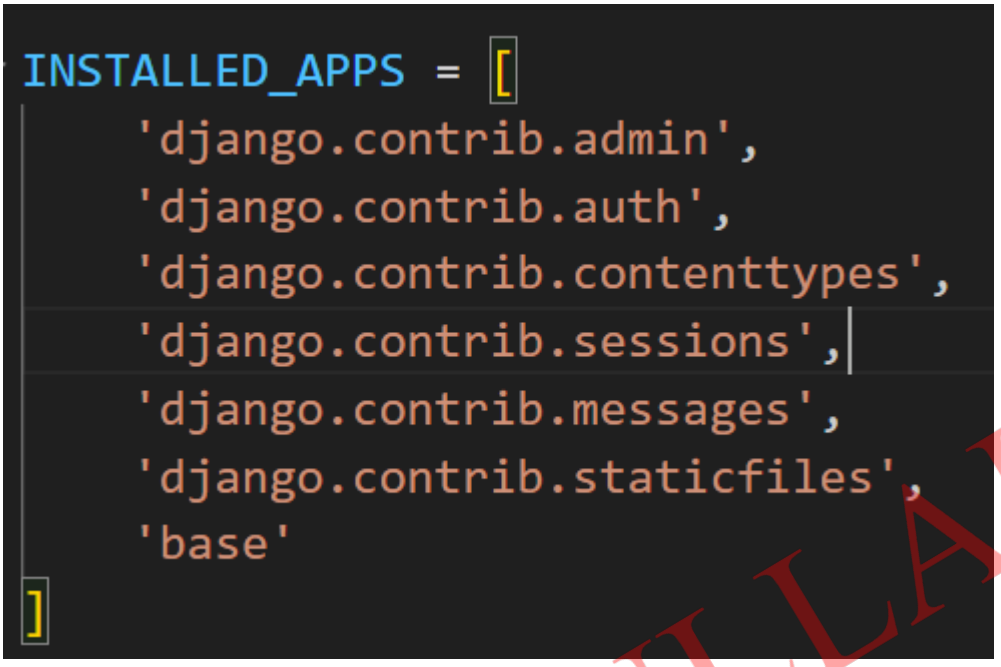
python manage.py startapp base

Important Files in a Django App

File	Purpose
apps.py	Registers the app configuration and allows Django to load the app
models.py	Defines database tables using Python classes
views.py	Contains application logic and handles HTTP requests and responses
admin.py	Registers models to be managed in Django Admin panel
tests.py	Used for unit testing the application
migrations/	Stores database schema changes
urls.py	Maps URLs to view functions (should be created inside each app)

Registering the App in the Project

After creating the app, it must be added in `settings.py`.



If the app is not registered:

- Models will not migrate
- URLs will not work
- Templates may not load properly

Including App URLs in Project

In the **project level** `urls.py`

```python
from django.urls import path,include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('',include('base.urls'))

]
```

Or

```python
from django.urls import path,include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('base/',include('base.urls'))
]
```


Purpose of `include()`

`include()` connects the **app URLs** with the **main project URLs**.

It helps Django route the request to the correct application.

Django URL Resolution Flow

1. A request comes from the **browser**.
2. Django first checks the **project-level** `urls.py` (main entry point).
3. The project `urls.py` finds the **matching path**.
4. Django then forwards the request to the **app-level** `urls.py`.
5. The **app** `urls.py` maps the URL to a **view function**.
6. The **view function (views.py)**:
    a. Executes business logic
    b. Interacts with the database if neede
    c. Prepare the response.
7. The **view returns an HTTP response**.
8. Django sends the response back to the **browser**, and the browser displays the output.

Simple Flow

Browser Request
↓
Project urls.py
↓
App urls.py
↓
views.py
↓
Database (optional)
↓
Response
↓
Browser Output